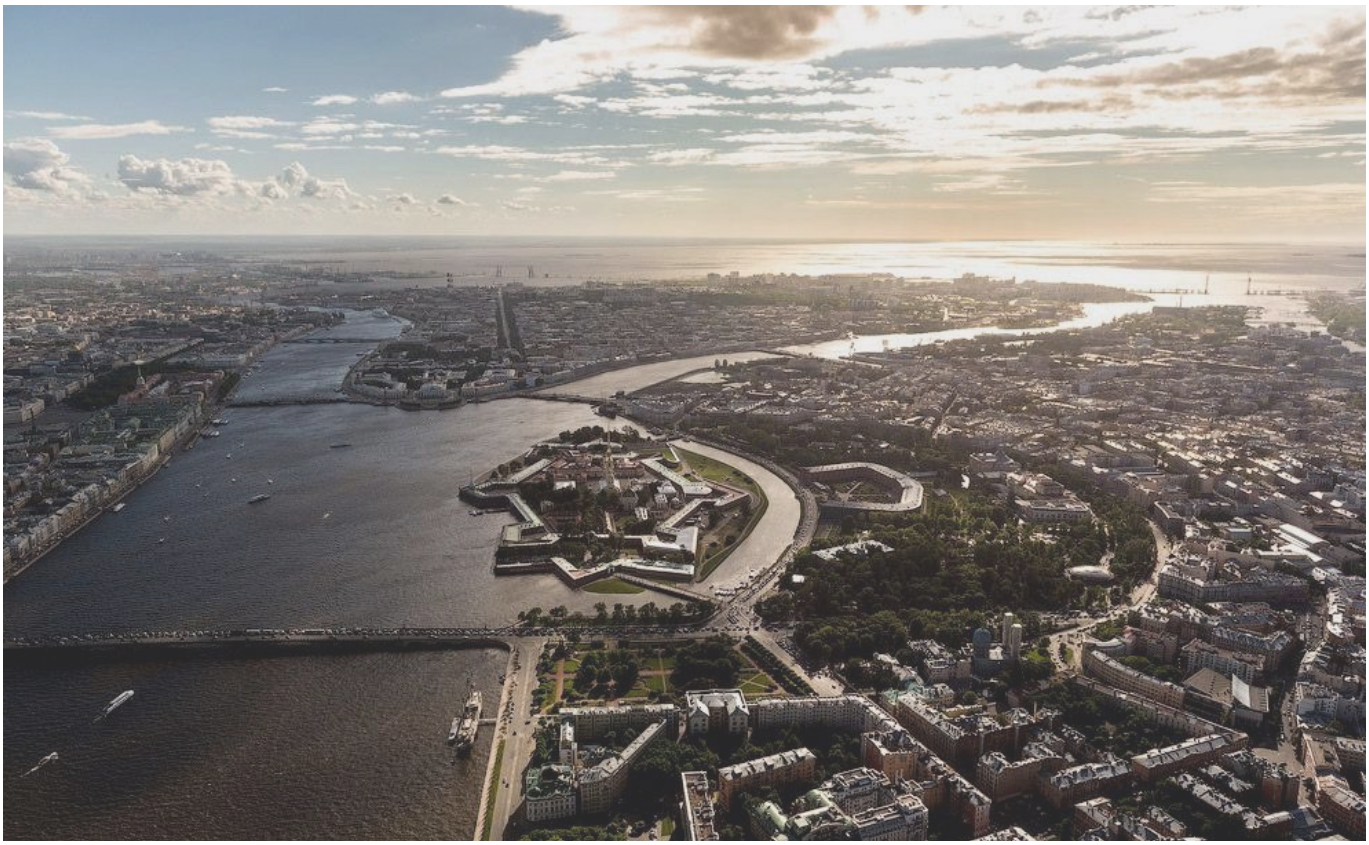
J3211, Сидельникова Дарья, 476010

ПРОЕКТИРОВАНИЕ БД

Тема 16: Поиск/покупка/аренда жилья

Лабораторная 5

(На примере https://cian.ru/)



Санкт-Петербург
2025 г

```
show wal_level;
show synchronous_commit;
show max_wal_size;
show min_wal_size;
show wal_compression;
show checkpoint_timeout;
show checkpoint_completion_target;
show archive_mode;
show archive_command;
```

```
wal_level| synchronous_commit|max_wal_size|min_wal_size| wal_compression|
---------+ -----------------+------------+------------+ ---------------+
replica  | on               |1GB         |80MB        | off            |

checkpoint_timeout|checkpoint_completion_target| archive_mode|
------------------+----------------------------+ ------------+
5min              |0.9                         | off         |

archive_command|
---------------+
(disabled)     |
```

Пояснение параметров:

wal_level = replica — определяет объём информации, записываемой в WAL
Значение replica позволяет использовать репликацию и архивирование

synchronous_commit = on — означает, что транзакция считается завершённой
только после записи данных в WAL

max_wal_size и min_wal_size — задают допустимый диапазон размера WAL-файлов

wal_compression = off — WAL не сжимается, что увеличивает его размер,
но снижает нагрузку на CPU

checkpoint_timeout — максимальный интервал между чекпоинтами

checkpoint_completion_target — определяет, какую часть времени между
чекпоинтами PostgreSQL будет использовать для записи данных на диск

archive_mode и archive_command — отвечают за архивирование WAL.
В данной конфигурации архивирование отключено

```sql
select
    pg_current_wal_lsn() as lsn,
    pg_walfile_name(pg_current_wal_lsn()) as wal_filename;
```

```
lsn       |wal_filename            |
---------+------------------------+
0/17AF0D8|000000010000000000000001|
```

```
du -ha /var/lib/postgresql/18/docker/pg_wal
```

```
root@2e95bb4b5232:/# du -ha /var/lib/postgresql/18/docker/pg_wal
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000001
4.0K    /var/lib/postgresql/18/docker/pg_wal/summaries
4.0K    /var/lib/postgresql/18/docker/pg_wal/archive_status
17M     /var/lib/postgresql/18/docker/pg_wal
```

```
ls /var/lib/postgresql/18/docker/pg_wal
```

```
root@2e95bb4b5232:/# ls /var/lib/postgresql/18/docker/pg_wal
000000010000000000000001  archive_status  summaries
```

```sql
create table test (
    id serial primary key,
    data text
);

insert into test(data)
select repeat('a', 100000)
from generate_series(1, 500000);

select
    pg_current_wal_lsn() as lsn,
    pg_walfile_name(pg_current_wal_lsn()) as wal_filename;
```

```
lsn        |wal_filename            |
----------+------------------------+
0/4C8C5410|000000010000000000000004C|
```

```
root@2e95bb4b5232:/# ls /var/lib/postgresql/18/docker/pg_wal
000000010000000000000029  000000010000000000000003A  000000010000000000000004B  000000010000000000000005C
000000010000000000000002A  000000010000000000000003B  000000010000000000000004C  000000010000000000000005D
000000010000000000000002B  000000010000000000000003C  000000010000000000000004D  000000010000000000000005E
000000010000000000000002C  000000010000000000000003D  000000010000000000000004E  000000010000000000000005F
000000010000000000000002D  000000010000000000000003E  000000010000000000000004F  000000010000000000000060
000000010000000000000002E  000000010000000000000003F  000000010000000000000050  000000010000000000000061
000000010000000000000002F  000000010000000000000040  000000010000000000000051  000000010000000000000062
000000010000000000000030  000000010000000000000041  000000010000000000000052  000000010000000000000063
000000010000000000000031  000000010000000000000042  000000010000000000000053  000000010000000000000064
000000010000000000000032  000000010000000000000043  000000010000000000000054  000000010000000000000065
000000010000000000000033  000000010000000000000044  000000010000000000000055  000000010000000000000066
000000010000000000000034  000000010000000000000045  000000010000000000000056  000000010000000000000067
000000010000000000000035  000000010000000000000046  000000010000000000000057  archive_status
000000010000000000000036  000000010000000000000047  000000010000000000000058  summaries
000000010000000000000037  000000010000000000000048  000000010000000000000059
000000010000000000000038  000000010000000000000049  00000001000000000000005A
000000010000000000000039  00000001000000000000004A  00000001000000000000005B
```

```
root@2e95bb4b5232:/# du -ha /var/lib/postgresql/18/docker/pg_wal
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000004E
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000065
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000003C
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000047
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000004B
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000058
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000029
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000045
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000004A
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000048
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000033
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000003A
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000005C
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000044
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000053
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000036
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000059
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000003D
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000035
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000060
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000005E
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000057
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000040
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000002F
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000003F
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000067
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000032
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000051
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000042
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000039
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000034
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000004C
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000005A
4.0K    /var/lib/postgresql/18/docker/pg_wal/summaries
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000043
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000030
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000002B
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000002D
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000005F
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000003B
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000064
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000041
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000055
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000002C
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000037
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000061
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000052
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000003E
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000066
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000004D
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000005D
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000046
16M     /var/lib/postgresql/18/docker/pg_wal/000000010000000000000054
16M     /var/lib/postgresql/18/docker/pg_wal/00000001000000000000002E
```

При массовой вставке данных объём WAL увеличился, а WAL-файл изменился.
Это связано с тем, что PostgreSQL записывает все изменения данных сначала в WAL для обеспечения надёжности и возможности восстановления после сбоя.

чекпоинты

```
insert into test(data)
select repeat('b', 100000)
from generate_series(1, 200000);

select * from pg_stat_checkpointer;
```

num_timed	num_requested	num_done	restartpoints_timed	restartpoints_req	restartpoints_done	write_time	sync_time	buffers_written	slru_written	stats_reset
8	3	4	0	0	0	372576.0	146.0	21805	9	2025-12-22 10:31:00.1

```
select
    pg_current_wal_lsn() as lsn,
    pg_walfile_name(pg_current_wal_lsn()) as wal_filename;
```

lsn	wal_filename
0/5F8F8780	000000010000000000000005F

```
checkpoint;
select * from pg_stat_checkpointer;
```

num_timed	num_requested	num_done	restartpoints_timed	restartpoints_req	restartpoints_done	write_time	sync_time	buffers_written	slru_written	stats_reset
8	4	6	0	0	0	551112.0	171.0	27608	10	2025-12-22 10:31:00.1

После выполнения команды checkpoint статистика чекпоинтов изменилась, что подтверждает принудительную запись изменённых данных на диск.
Чекпоинты позволяют ограничивать объём WAL и ускоряют восстановление после сбоя.

Изменение конфига + перезапуск

```
# - Checkpoints -

#checkpoint_timeout = 1min
```

```
insert into test(data)
select repeat('c', 100000)
from generate_series(1, 100000);

select * from pg_stat_checkpointer;
```

```
num_timed|num_requested|num_done|restartpoints_timed|restartpoints_req|restartpoints_done|write_time|sync_time|buffers_written|slru_written|stats_reset
---------+-------------+--------+------------------+----------------+------------------+----------+---------+---------------+------------+---------------------
       12|            5|       6|                 0|               0|                 0|  551113.0|    172.0|          27608|          10|2025-12-22 10:31:00.1
```

Спустя несколько минут…

```
num_timed|num_requested|num_done|restartpoints_timed|restartpoints_req|restartpoints_done|write_time|sync_time|buffers_written|slru_written|stats_reset                  |
---------+-------------+--------+------------------+----------------+------------------+----------+---------+---------------+------------+-----------------------------+
       13|            5|       6|                 0|               0|                 0|  551113.0|    172.0|          40134|          13|2025-12-22 10:31:00.110 +0300|
```

Уменьшение checkpoint_timeout приводит к более частым чекпоинтам, что увеличивает нагрузку на диск, но уменьшает объём WAL. При этом восстановление после сбоя происходит быстрее, так как требуется обработать меньше WAL-данных.

Бэкап

```
root@2e95bb4b5232:/# ls /var/lib/postgresql/base_backup
backup_label     pg_commit_ts   pg_logical    pg_serial     pg_subtrans  pg_wal
backup_manifest  pg_dynshmem    pg_multixact  pg_snapshots  pg_tblspc    pg_xact
base             pg_hba.conf    pg_notify     pg_stat       pg_twophase  postgresql.auto.conf
global           pg_ident.conf  pg_replslot   pg_stat_tmp   PG_VERSION   postgresql.conf
```

Физический бэкап содержит полную копию кластера PostgreSQL, включая данные, конфигурационные файлы и WAL.
Параметр -X stream означает, что WAL передаётся во время создания бэкапа, что обеспечивает его консистентность.
Формат каталога (-Fp) представляет бэкап в виде файловой структуры, в отличие от формата tar (-Ft), который создаёт архив.

```
insert into test(data) values ('new data after backup');
select count(*) from test;
```

```
count |
------+
800001|
```

```sql
create database postgres_restored;
select count(*) from test;
select * from test where data = 'new data after backup';
```

```
id    |data                  |  count |
------+----------------------+  ------+
800001|new data after backup|  800001|
```

Физический бэкап (pg_basebackup) копирует весь кластер PostgreSQL и используется для полного восстановления сервера.
Логический бэкап (pg_dump) сохраняет структуру и данные отдельных баз, что позволяет восстанавливать их выборочно или на другой версии PostgreSQL.
Физический бэкап быстрее, но менее гибкий. Логический бэкап медленнее, но удобнее для миграций.